

Project 1: Rotary Encoder Driver

Author: Eiríkur Bernharðsson

Board: Arduino Nano (ATmega328P, 16 MHz)

Motor: JGA12-N20 gearmotor, 1:100 gearbox, with a Hall effect quadrature encoder

Video: https://youtu.be/Dz__JQw9IRg

Repository: https://github.com/eirikur7/EmbeddedSystems_Project1

Summary

	Calculated	Measured
Encoder resolution	1400 counts per revolution	~1400 (turned by hand)
Maximum pulse rate	3617 edges/s (1808 Hz signal)	3604 edges/s (1802 Hz)
Must sample faster than (counting)	every 276 μ s	fails above 277 μ s
Must sample faster than (direction)	every 138 μ s	fails above ~139 μ s
Response time, polled	$\leq 100.7 \mu$ s	59-96 μ s
Response time, interrupt	-	2.2-7.9 μ s

The measured signal matches the calculation from the datasheet to within 0.3 %.

1. Setup

Wiring

Pin	Use
D2	Encoder C1 - the channel that is counted. Also the INT0 interrupt pin, used in Part 3
D3	Encoder C2 - read at each C1 edge to work out the direction
D4	External LED, pulsed once per handled state change
D0, D1	Serial to the laptop, 9600 baud

All four pins are on port D, which is what the `Digital_in` and `Digital_out` classes from A1.2 use. C1 is on D2 specifically because that pin has the INT0 interrupt, which Part 3 needs.

No timers are used anywhere. Sample timing in Parts 1 and 2 comes from `_delay_us()`, which is just a counting loop.

Measurement setup

- Motor run from 6 V with nothing attached to the output shaft.
- Oscilloscope: Rohde & Schwarz RTB2004, 10:1 probes, DC coupled.
 - Ch1 (yellow) = C1, Ch2 (green) = C2, Ch3 (orange) = LED on D4.
- Measured: C1 pulse width, LED pulse width, C1→LED delay, C1→C2 phase.
- Serial output read in the PlatformIO terminal at 9600 baud.

Running with no load matters, because the datasheet gives two different speeds and the no-load one is the faster of the two.

2. Part 1: Polled driver

Encoder resolution

The encoder gives 7 pulses per turn of the motor's own shaft, and the gearbox is 1:100, so one turn of the output shaft is:

$$7 \times 100 = 700 \text{ square wave cycles on C1}$$

The driver counts both rising and falling edges, and each cycle has two edges:

$$700 \times 2 = 1400 \text{ counts per revolution of the output shaft}$$

Resolution is 1400 counts per revolution. This is fixed by the hardware and does not change with speed.

Maximum pulse rate

To turn that into a rate, the fastest shaft speed is needed. The datasheet gives two speeds at 6 V: **155 RPM with no load** and **90 RPM at rated torque**. The maximum pulse rate comes from the no-load figure, since that is as fast as the shaft can go.

At 155 RPM the shaft turns 2.583 times per second:

$$\begin{aligned} \text{edges per second} &= 1400 \times 2.583 = 3617 \text{ edges/s} \\ \text{signal frequency} &= 700 \times 2.583 = 1808 \text{ Hz} \end{aligned}$$

$$\text{period} = 1 / 1808 = 553 \mu\text{s}$$

The maximum pulse rate on C1 is 3617 edges per second, carried on a 1808 Hz square wave.

Note that these are two different numbers. Each cycle of the square wave contains two edges, so the edge rate is always twice the frequency. Mixing them up makes every timing figure that follows wrong by a factor of two.

How often the encoder has to be read

There are two separate requirements, and they are not the same number.

To avoid missing a pulse. The driver spots a change by comparing C1 now against C1 at the last sample. So it sees every change as long as it takes at least one sample while C1 is high and at least one while C1 is low. The rule is:

$$\text{time between samples} < \text{length of the shortest high or low state}$$

At a 50 % duty cycle that state is half a period, $553 / 2 = 276 \mu\text{s}$.

To get the direction right. Direction is not in either channel on its own. It is in the phase between the two, and that changes twice as often as either wire does. Treating C1 and C2 as a two-bit value, it walks through four states per cycle, and the order tells you the direction:

```
forward:  (0,0) → (1,0) → (1,1) → (0,1) → (0,0)
reverse:  (0,0) → (0,1) → (1,1) → (1,0) → (0,0)
```

Each state lasts a quarter of a cycle, so the pair changes every $553 / 4 = 138 \mu\text{s}$, with the edges alternating between the channels:

```
t = 0 μs    C1 edge
t = 138 μs   C2 edge
t = 276 μs   C1 edge
t = 415 μs   C2 edge
t = 553 μs   C1 edge (repeats)
```

To follow a sequence that moves every $138 \mu\text{s}$, one sample per state is needed, so four per cycle.

Requirement	Max time between samples	Minimum rate
Do not miss a pulse	276 μs	3617 Hz
Get the direction right	138 μs	7233 Hz

Direction is the stricter of the two and is what sets the design.

The same 138 μs comes out of the code. `update()` reads C2 the moment it notices a C1 edge, and C2 flips 138 μs after that edge. Notice the edge any later and C2 has already changed, so the direction comes out backwards:

```
t = 0  $\mu\text{s}$     C1 rises, C2 still low
t = 138  $\mu\text{s}$    C2 rises
t = 140  $\mu\text{s}$    sample happens, C2 is read but it flipped 2  $\mu\text{s}$  ago
                  update() sees both high, so it decrements instead of
incrementing
```

Importantly, the C1 edge is still seen in that case. Only the sign is wrong. That gives a prediction to test on the scope: direction should start failing around 138 μs , while counts only start disappearing past 276 μs .

Implementation

`Encoder ( encoder.h / encoder.cpp )` holds two `Digital_in` objects, one per channel, and exposes `int position()`. `update()` checks C1 for a change and, when it finds one, reads C2 to decide whether to add or subtract. All pin access goes through `Digital_in`, so `encoder.cpp` has no registers in it and the header shows no hardware detail at all.

The main loop calls `update()` with `_delay_us(100)` between samples, about 10 kHz. Against the two limits above that gives 2.7 $\times$ margin on counting and 1.4 $\times$ on direction. The direction margin is the one that matters, and 138 μs is where it breaks rather than where it works, so some room below it is needed. Sampling faster costs nothing here because the processor has no other job.

An LED on D4 goes high for one loop pass whenever `update()` reports a change, and low otherwise. That pin is the scope probe point.

Scope measurements

Direction. The motor was run one way, then the two motor leads were swapped to run it the other way. The phase between C1 and C2 changes sign:

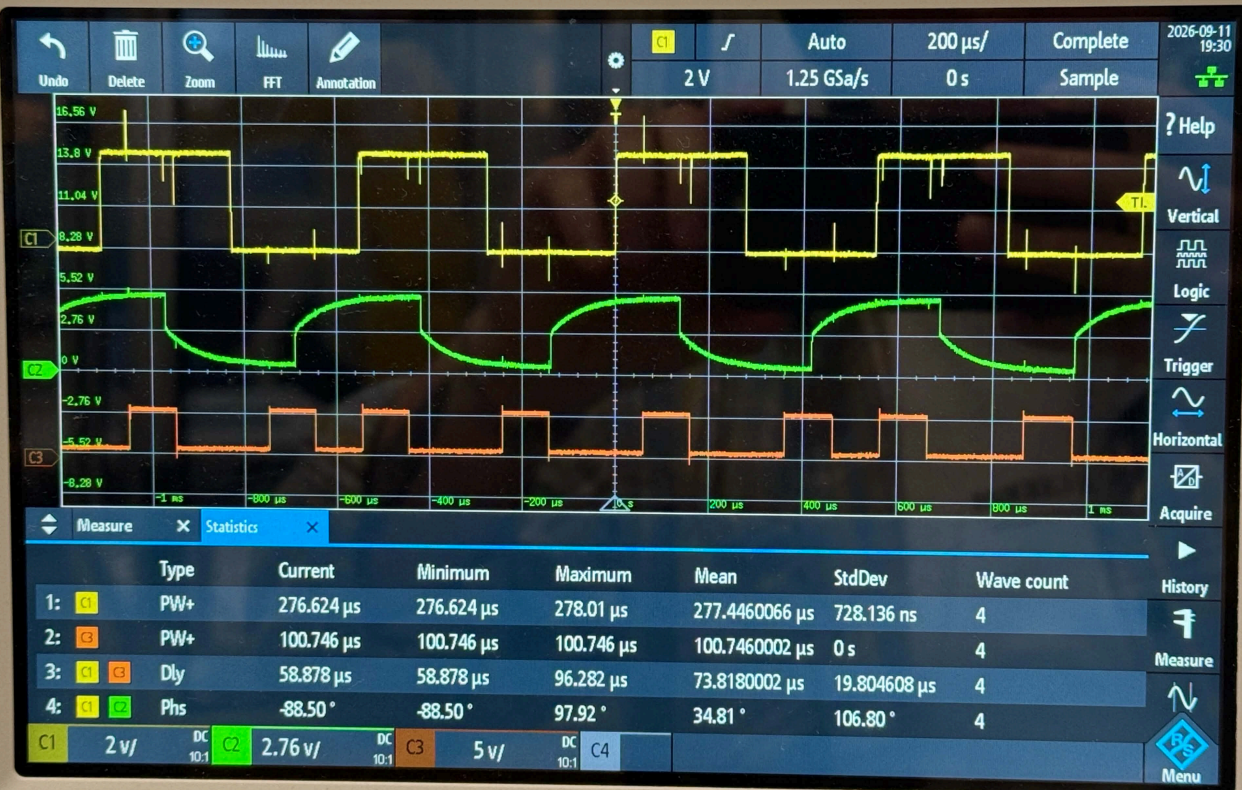
Motor leads	C1→C2 phase
negative on M2	-88.50°
positive on M2	+97.92°



ROHDE & SCHWARZ

RTB2004 · Digital Oscilloscope · 2.5 GSa/s

OSC 12107699



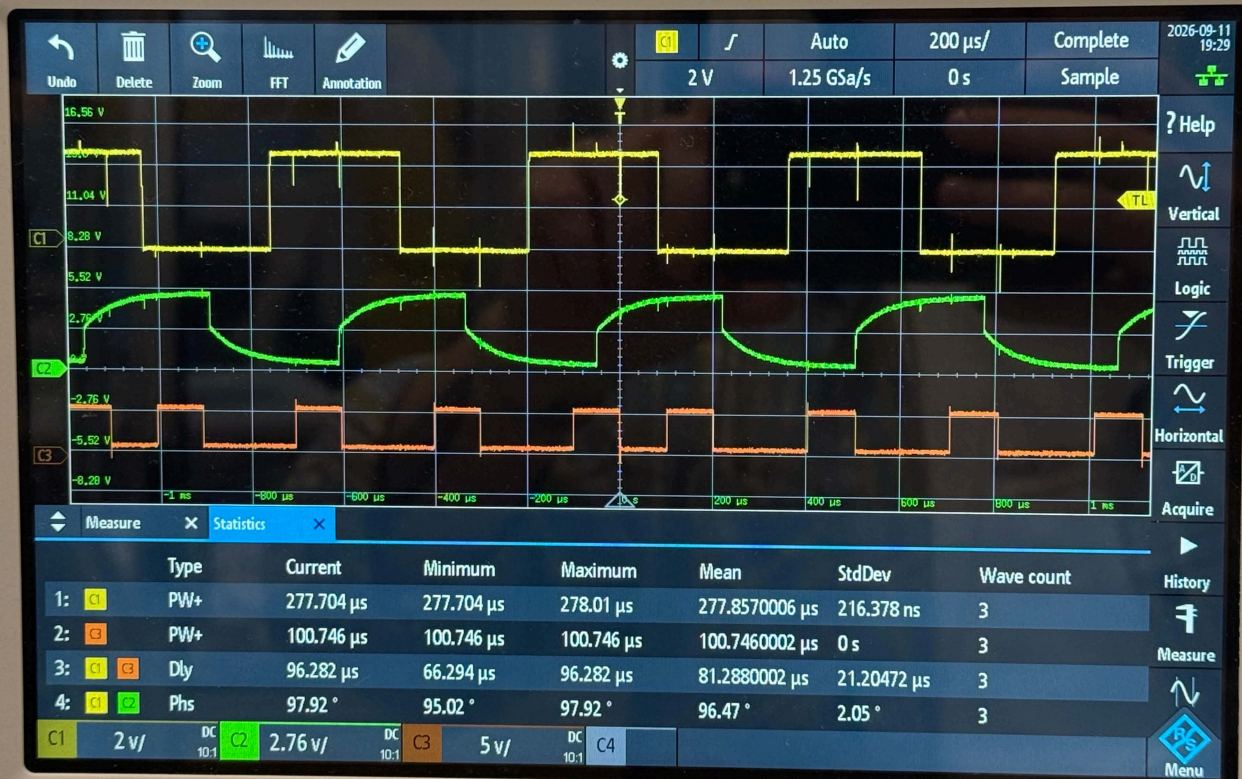
A223.02-0319



ROHDE & SCHWARZ

RTB2004 · Digital Oscilloscope · 2.5 GSa/s

OSC 12107699



The sign flip is the direction reversal. Both are close to 90°, which confirms the channels really are a quarter cycle apart as assumed above. The statistics were not reset between the two runs, so only the Current column means anything here.

Speed. C1 positive pulse width measures **277.4 µs**, so a full period is about 555 µs, or 1802 Hz.

This is the main confirmation of the calculation. The 155 RPM figure predicted a 276.5 µs state; the measurement is 277.4 µs, which is 0.3 % off. Working backwards it corresponds to 154.5 RPM against a 155 RPM specification.

An earlier version of this calculation used the 90 RPM rated speed instead, which predicted 476 µs and disagreed with the measurement by a factor of 1.7. The rated speed is the speed under load; with the shaft spinning free the no-load speed is the right number to design against.

Loop period. The LED is high for exactly one loop pass, so its pulse width is the loop period. It measures **100.746 µs** with no variation between captures. That is the `_delay_us(100)` plus about 0.75 µs of work, so `update()` itself costs almost nothing.

Response time. The delay from a C1 edge to the LED going high ranged from **59 µs to 96 µs**, averaging around 75-80 µs. This is how long it takes between the encoder changing and the code having dealt with it.

The spread is expected. An edge can arrive anywhere inside a sample period, so the response is anything from near zero up to one full loop period. Every measurement falls inside the 100.746 µs loop period, which is the predicted bound.

Sweeping the sample rate

The loop delay was raised to find the two failure points predicted above: 138 µs for direction and 277 µs for counting.

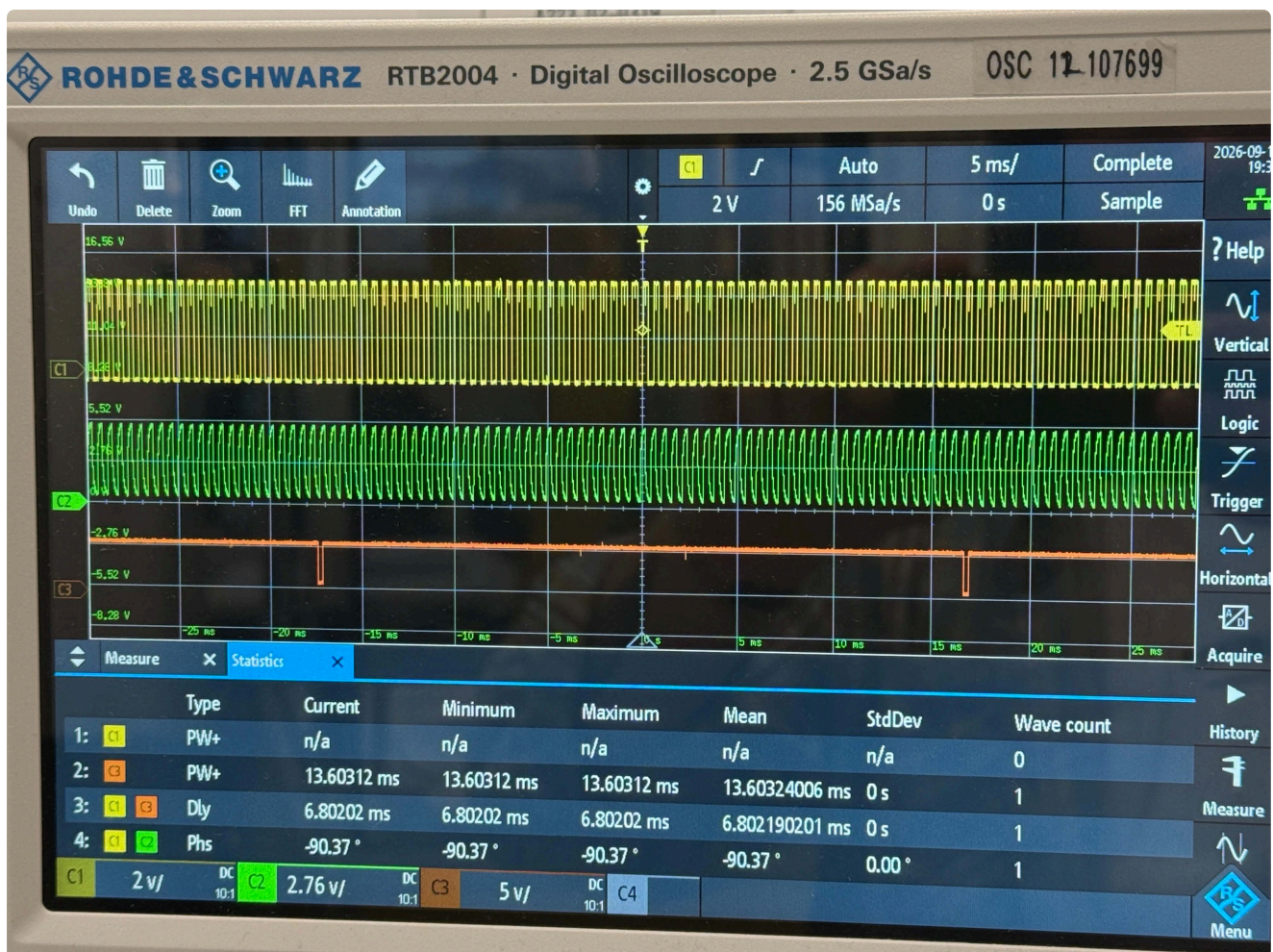
200 µs - between the two limits.



C1 is unchanged at 277 μs and every C1 edge still produces an LED pulse, so nothing is being missed. The LED now stays high for 401.9 μs, which is two loop passes: with a 200 μs loop and edges 277 μs apart, an edge sometimes falls into two consecutive sample windows.

Counting still works here, but the response time is now up to 200 μs, well past the 138 μs direction limit. This is the case that shows the two failures are separate, the scope looks fine while the direction is already wrong.

278 μs - at the counting limit.



Here the loop period is almost exactly the 277 μ s edge spacing. Every sample lands one edge further on than the last, so C1 looks different every time, `update()` reports a change on nearly every pass, and the LED sits high continuously instead of pulsing, 13.6 ms of continuous high between dips.

The two rates are not quite equal, so the timing drifts slowly. Every so often two samples land in the same half cycle, no change is seen, and the LED drops for one pass. Those are the narrow dips.

This is aliasing. The rate of detected changes looks about right, which is why the LED is almost always on, but the count itself is a different story.

Count rate against sample period

The motor was run at 6 V in one direction and the serial output recorded for 10 seconds at each setting.

Delay	Start	End	Counts in 10 s	Rate
100 μ s	-3824	27011	34 700	3470 /s
200 μ s	-1291	-15292	14 001	1400 /s
278 μ s	-1301	-5715	4 414	441 /s

The count decreases in all three runs. The 100 μ s run passed through the bottom of the 16-bit range on the way and reappeared at the top, so its total is the two segments added together: 28 944 counts from -3824 down to -32768, then 5756 more from 32767 down to 27011.

The values printed either side of that wrap were -32745 and 32068, which are 722 counts apart. One print interval at this rate is $2000 \text{ samples} \times 100.746 \mu\text{s} \times 3470 \text{ counts/s} \approx 699$ counts, so the gap is one print long, as it should be. That is a useful check that no counts went missing across the wrap.

The 100 μ s run is slightly below the true edge rate, and the serial print explains it. The scope gives 3604 edges/s while this run counts 3470/s, about 4 % lower. Every edge is caught and every direction is right at this sample period, so it is not a sampling failure. It is the print: with `print_interval` at 2000 the loop prints every 201 ms, and each line blocks the loop for 7-8 ms while it is sent. That is 3.5-4 % of the time with no sampling at all, and the edges during a stall collapse into at most one count. Predicted 3460-3480 /s, measured 3470 /s.

So even the deliberately conservative Part 1 setup loses a few percent to the serial link. Part 2 is what happens when that mitigation is removed.

Why the rate collapses as the delay grows. The response time is spread evenly from zero up to one loop period, and the direction is wrong whenever it is longer than 138 μ s. The share of counts with the wrong sign is therefore $(T_s - 138) / T_s$ once T_s is past 138 μ s, and zero below it. A wrong count subtracts instead of adding, so the net rate is scaled by:

```
Ts ≤ 138 μs : factor = 1
Ts > 138 μs : factor = (277 - Ts) / Ts
```

Delay	Predicted factor	Predicted rate	Measured rate
100 μ s	1.00	3470 /s	3470 /s
200 μ s	0.39	1350 /s	1400 /s
278 μ s	0.00	≈ 0	441 /s

The 200 μ s result is within about 4 % of the prediction, which is good for a model this simple. At 278 μ s the delay is almost exactly twice the direction limit, so half the edges should be counted backwards and the count should nearly stand still. It nearly does, at 13 % of the true rate. It is also unstable: the step between printed values jumped between roughly 100 and 700, because a fraction of a percent of speed drift tips the balance either way.

The point: no edges are lost at 200 μ s or 278 μ s. The counting still works. What fails is the direction, and wrong directions subtract rather than leaving a gap. A slow count does not mean missed pulses, it means the driver is fighting itself. Past 277 μ s the factor goes

negative, so the count should run backwards while the shaft turns forwards. This was not tested.

Verification

Direction. Running the motor one way makes the count climb steadily; swapping the motor leads makes it fall. So the counter follows the direction of rotation, not just the presence of pulses.

Counting. The output shaft was marked, the count noted, the shaft turned one full revolution by hand, and the count noted again. It came out at roughly 1400 every time.

By construction the answer is exactly 1400, and turning by hand cannot be precise, judging a full revolution by eye is only good to a few degrees, and a few degrees is several counts at this resolution. The check is not there to measure the resolution, it is there to confirm the $7 \times 100 \times 2$ calculation and rule out being out by a factor of two. It does that.

3. Part 2: Printing the count over UART

The count is printed with the Arduino `Serial` class at 9600 baud. In Part 1 it printed only once every 2000 samples so the serial link stayed mostly out of the way. For Part 2 `print_interval` was set to 1 so every sample prints. Nothing else changed.

The serial link takes over the timing



The LED pulse width is the loop period. In Part 1 it was 100.746 μ s. Here it is **7.2895 ms**, which is **72 times longer**.

That number is not a coincidence. At 9600 baud each character takes 10 bit times, or 1.0417 ms, and a line is 7 characters:

$$7 \times 10 / 9600 = 7.2917 \text{ ms} \quad (\text{predicted})$$

$$7.2895 \text{ ms} \quad (\text{measured})$$

So the loop period is now just the time to push one line out of the serial port, and `_delay_us(100)` is irrelevant. A side effect worth noting is that the loop period now depends on how many digits the number has, so the sample rate changes as the count grows.

Nothing is wrong with the encoder itself. C1 still measures 279.8 μ s and the phase is still near -86 $^{\circ}$, so the signal at the pins is unchanged. Only the ability to look at it has broken.

Effect on the counting

7.2895 ms is 26 times the 277 μ s counting limit and 53 times the 138 μ s direction limit, so both are broken by a wide margin.

	Part 1 (every 2000 samples)	Part 2 (every sample)
Loop period	100.746 μ s	7.2895 ms

	Part 1 (every 2000 samples)	Part 2 (every sample)
Count rate	3470 /s	roughly 20 /s

Two different failures were seen on different runs, which is itself the interesting part.

In one run the count simply wandered, moving up and down by one between samples and staying between about 6 and 13 for the whole recording. It never accumulated at all.

In another run the count climbed steadily but absurdly slowly, roughly 200 counts in 10 seconds where it should have been about 36 000. (This figure is approximate - it was read off the terminal rather than taken from start and end values.)

Both come from the same cause. Once the sample period is longer than a whole encoder cycle, the level of C2 at sampling time has no fixed relationship to which C1 edge is being seen, so the direction decision is close to random. Whether the errors cancel out, giving a wander, or lean slightly one way, giving a slow drift, depends on accidental relationships between the baud rate, the number of digits and the motor speed. Neither result says anything about the shaft. Both are shown in the video.

The response time measurement also stops meaning anything. In Part 1 it was tightly bounded between 59 and 96 μ s; here it reads anywhere from -35 ms to +17 ms, because the LED edge no longer belongs to any particular C1 edge.

Conclusion for Part 2

The serial output does interfere with the counting, and at 9600 baud printing every sample it does not just degrade the measurement, it destroys it. The driver is unchanged from Part 1 and still correct. What changed is that a blocking serial write was put inside the sampling loop, and sending one line takes about 70 times longer than the whole Part 1 loop.

Printing once every 2000 samples, as in Part 1, works because it spreads that cost over many samples - but as shown above it does not remove it, it only reduces it to about 4 %. The loop still stops for 7 ms whenever a print happens. Part 3 fixes the underlying problem.

4. Part 3: Reading the encoder from an interrupt

Implementation

C1 is on D2, which is the INT0 pin, so no pin change interrupt is needed. The driver sets INT0 up itself in `init()`:

```
EICRA |= (1 << ISC00);    // any change, so both edges of C1 trigger
EICRA &= ~(1 << ISC01);
```

```
EIMSK |= (1 << INT0);    // enable INT0
sei();                   // enable interrupts globally
```

Triggering on any change rather than a single edge keeps the resolution at 1400. Triggering on one edge would halve it to 700. The global enable is needed explicitly because `main()` does not call the Arduino core's own `setup`.

Three details were needed to make it work:

- An interrupt handler is a plain function with a fixed name, so it cannot be a class member. `init()` stores the instance in a file-scope pointer and `ISR(INT0_vect)` calls through it. The hardware knowledge stays in the `.cpp`.
- `count` is written by the interrupt and read by the main loop, so it is `volatile`. Without that the compiler could keep it in a register and `position()` would return a stale value forever.
- `count` is 16 bit, so reading it takes two instructions. `position()` turns interrupts off across the read and restores the previous state afterwards, so an interrupt cannot land in the middle and return half of an old value.

The handler is simpler than the polled version, because the hardware has already found the edge - there is no previous level to compare against.

The LED moved inside the driver in this part. It goes high on entry and low on exit, so its rising edge is the response time and its width is how long the handler took, both readable straight off the scope.

`main()` is now just a loop that prints. There is no delay.

One limitation: the constructor takes `pin_c1` as an argument but `init()` hard-codes INT0, so the driver only works with C1 on D2. The header notes this.

Measurements



Statistics over 726 acquisitions:

	Part 1 (polled)	Part 3 (interrupt)
Response time, C1 edge to LED high	59-96 μ s	2.20-7.85 μs , mean 2.53
Handler run time	-	2.12 μs

Mean response is about 30 times better, and even the worst case of 7.85 μ s beats the best case of the polled version by a factor of seven. That worst case is the serial port's own interrupt briefly blocking this one, since AVR interrupts do not nest by default. A few microseconds against a 277 μ s edge spacing does not matter.

At 2.12 μ s per edge and 3604 edges per second the encoder costs about **0.8 %** of the processor.

Comparison with Part 2

The main loop prints on every pass at 9600 baud with no delay - the same serial load that destroyed the count in Part 2.

	Part 1	Part 2	Part 3
Sampling	polled, 100.7 μ s	polled, 7.29 ms	interrupt
Printing	one line per 2000 samples	every sample	every sample
Response time	59-96 μ s	meaningless	2.2-7.8 μ s
Counts in 10 s	34 700 (measured)	roughly 200 (estimated)	same order as Part 1 (estimated)

Moving the encoder read into an interrupt separates it from whatever the main loop is doing, so a blocking serial write no longer affects the sampling. This is the result the part asks for.

The strongest evidence is the response time rather than the count. It was measured over 726 acquisitions and sits between 2.20 and 7.85 μ s, which is far inside every limit derived in Part 1 comfortably below both the 276 μ s counting limit and the 138 μ s direction limit, with three orders of magnitude to spare. On those numbers alone no edge can be missed and no direction can be misread, whatever the main loop is busy with.

The count behaves accordingly. Instead of wandering between 6 and 13 as it did in Part 2, it accumulates steadily at roughly the Part 1 rate. That last figure was read off the terminal rather than taken from start and end values, so it is quoted as agreement in order of magnitude rather than as a precise match.

An unexpected result: interrupts pick up noise that polling ignored

The first attempt did not work. The count ran away, wrapping repeatedly through the 16-bit range where a correct run would barely wrap once.

A capture of the serial output from this run (`serial_part3_noisy.txt` , in the repository) holds 796 printed values, and it makes the failure quantitative. The count decreases steadily at a mean of 313 counts per printed line. The loop is serial-bound, so each line takes $(\text{digits} + 2) \times 10$ bit times at 9600 baud; summing that across the capture gives 5.85 s and a mean print interval of 7.34 ms, which agrees with the 7.2895 ms measured directly on the scope in Part 2. Over those 5.85 s the count travelled 249 213 counts:

```
249 213 counts / 5.845 s = 42 600 counts/s
true edge rate           = 3 604 counts/s
                        → 11.8 times too many
```

Because the count is the net of increments and decrements, 42 600/s is a lower bound on the interrupt rate any spurious interrupts that happened to cancel are invisible in it.

The endpoints of the full run agree independently. Going from -22626 down to -3264 over ten seconds, with seven wraps in between, is 439 000 counts or 43 900 counts/s, within 3 % of the figure from the capture.

One subtlety about counting wraps, since it is easy to get wrong by a factor of two: in a count that moves steadily in one direction the printed value changes sign **twice** per wrap, once at the wrap itself and once as it passes through zero. The capture confirms this exactly 8 sign changes across 4 wraps. The fourteen sign changes observed over the ten second run are therefore seven wraps, not fourteen.



The cause is visible on the scope. During each low period of C1 there is a burst of electrical noise swinging roughly ± 2.75 V, far more than enough to cross the input threshold several times. Directly below it the LED trace shows a dense packet of pulses, one per spurious interrupt. The hardware was correctly reporting edges that were not encoder transitions.

Polling never had this problem. The glitches are well under a microsecond, so sampling every 100 μ s almost never landed on one. The polled driver filtered them out by accident rather than by design.

The noise turned out to be **direction dependent**. Running the motor the other way gave the clean capture above, one interrupt per edge and a correct count. This is a known property of brushed DC motors: the brushes are set at an angle suited to one direction, and running against it makes the sparking worse.

This is a workaround, not a fix. The driver is still sensitive to sub-microsecond glitches; the condition that causes them was avoided rather than the sensitivity removed. A proper fix would filter the signal a 0.1 μ F capacitor across the motor terminals to cut the brush noise at the source, or an RC low-pass on C1 (1 k Ω and 10 nF gives a 10 μ s time constant, which flattens the glitches and leaves the 277 μ s edges alone).

The general lesson is that polling and interrupts do not just differ in speed. Polling looks at the signal at fixed moments and ignores anything short by accident, while an edge-triggered interrupt reacts to every threshold crossing and needs a clean input. Making the sampling 30 times faster also made it far more willing to believe noise.

5. Resource use

Build	Flash (of 30720 B)	RAM (of 2048 B)
Part 1	1450 B (4.7 %)	179 B (8.7 %)
Part 2	1440 B (4.7 %)	179 B (8.7 %)
Part 3	1584 B (5.2 %)	181 B (8.8 %)

Most of the RAM is the Arduino serial library's two 64-byte buffers; the driver itself uses only a few bytes.

The differences are small but they make sense. Part 2 is 10 bytes smaller than Part 1 even though the source is otherwise identical, because comparing against `print_interval = 1` is cheaper than loading and comparing the 16-bit constant 2000. Part 3 is about 140 bytes larger, which is the interrupt vector entry, the handler's register save and restore, and the interrupt-safe read in `position()`. Its 2 extra bytes of RAM are exactly the static pointer that lets the interrupt handler reach the `Encoder` object.

Processor time is the more interesting figure and differs sharply between the two designs:

	Part 1 / Part 2 (polled)	Part 3 (interrupt)
Time spent on the encoder	about 0.7 %	about 0.8 %
Processor left for anything else	none	about 99 %

The polled version barely touches the encoder, but `_delay_us()` keeps the processor busy doing nothing in between, so no other work can run. The interrupt version costs a similar number of microseconds per edge and leaves the rest of the processor free. That is a reason to prefer it quite apart from the timing.

6. Conclusion

The encoder gives 1400 counts per revolution of the output shaft. At the 155 RPM no-load speed that is 3617 edges per second on a 1808 Hz square wave, and the scope confirms this to within 0.3 %.

Two separate timing requirements follow, and telling them apart is the main trap in this project. Not missing a pulse needs a sample every 276 μ s. Getting the direction right needs a sample every 138 μ s. Direction is the stricter one, and the sweep confirms both boundaries independently: at 200 μ s the count still works while the direction is already wrong, and only past 277 μ s does the counting itself break.

The polled driver at 100 μ s meets both, but it has two costs that are easy to miss. It keeps the processor busy full time, and the periodic serial print still costs about 4 % of the counts even at one line per 2000 samples. Removing that mitigation in Part 2 drops the count to well under 1 % of the true rate not because the driver is wrong, but because a blocking serial write inside the sampling loop stretches the sample period by a factor of 72.

The interrupt version in Part 3 removes both problems. Response time improves roughly thirtyfold, the count survives the heaviest serial load, and almost the whole processor is free. The one thing it introduces is sensitivity to electrical noise, which polling had been hiding by accident. That was worked around rather than solved, and the proper fix is filtering the input.

Appendix: code layout

The submission takes individual files rather than an archive, so the per-part folders are flattened here. Files that are identical in every part keep their name; the ones that differ carry a part suffix.

File	Used by
<code>digital_in.h</code> , <code>digital_in.cpp</code>	All parts, unchanged from A1.2
<code>digital_out.h</code> , <code>digital_out.cpp</code>	All parts, unchanged from A1.2
<code>encoder_part1_2.h</code> , <code>encoder_part1_2.cpp</code>	Parts 1 and 2, polled driver
<code>encoder_part3.h</code> , <code>encoder_part3.cpp</code>	Part 3, interrupt driver
<code>main_part1.cpp</code>	Part 1, prints every 2000 samples
<code>main_part2.cpp</code>	Part 2, prints every sample
<code>main_part3.cpp</code>	Part 3, prints continuously

Parts 1 and 2 differ only in the value of `print_interval` . In the git repository the files sit in `results/part1/` , `results/part2/` and `results/part3/` under their plain names (`encoder.h` , `encoder.cpp` , `main.cpp`), the working copy in `src/` and `include/` is the Part 3 version, and the serial capture used in Part 3 is there as `results/report/serial_part3_noisy.txt` .